Лекція 4. Class design, Polymorphism
План:

1. **Пакети, імпорт пакетів.**
2. **Модифікатори доступу public та default.**
3. **Інтерфейси, реалізація інтерфейсів.**
4. **Низхідне та висхідне приведення об'єктів.**
5. **Концепція пізнього зв'язування.**

У мові Java *пакети* (*packages*) відіграють таку ж саму роль, як бібліотеки в інших мовах програмування. Пакети – це контейнери, які використовуються для ізольованого зберігання споріднених класів з метою уникнення перетину імен класів. Для того, щоб отримати доступ до класів, які входять у пакет, його необхідно імпортувати.

Файл вихідних пакетів у мові Java може налічувати будь-які (або усі) перераховані нижче складові:

1) один оператор пакету (не обов'язково);

2) довільну кількість операторів імпорту (не обов'язково);

3) одне оголошення загальнодоступного класу (обов'язково);

4) довільну кількість класів з модифікатором доступу `private` у пакеті (не обов'язково).

У розглянутих вище прикладах не було необхідності у використанні пакетів. Завжди існував тільки загальнодоступний клас, а імена всіх класів належали одному і тому ж *простору імен* – пакету по замовчуванню.

Створення пакету

Усі компоненти програми, записані в окремих файлах з розширенням `.java` та `.class`, які функціонально зв'язані між собою, рекомендується поміщати у окремий пакет. Для створення пакету необхідно на початку кожного файлу з вихідним кодом програми прописати наступний оператор:

package ім'я_пакету;

Усі класи, оголошені в цих файлах, будуть занесені до пакета з іменем `ім'я_пакету`. Якщо оператор `package` відсутній у файлі, то імена класів у цьому файлі, за замовчуванням належать до пакету без імені.

Для зберігання пакетів у мові Java використовують систему імен каталогів. Наприклад, якщо створити пакет з назвою `package mypackage`, то всі файли з розширенням *.class,* які належать до `mypackage`, повинні перебувати у каталозі з іменем *mypackage.* Важливо, щоб ім'я директорії точно збігалося з іменем пакета.

Можна створювати ієрархію пакетів, використовуючи для цього наступний синтаксис:

package пакет_1[.пакет_2 . . . [.пакет_N]];

Вкладеність пакетів у мові Java обмежується тільки можливостями файлової системи. Ієрархія пакетів повинна відображатися в структурі директорій. Пакет, оголошений як *package java.awt.image* передбачає, що у каталог із назвою `java` поміщений каталог з назвою `awt` у якому є каталог з назвою *image,* який у свою чергу включає файли із розширенням `.`*class.* Для того, щоб до пакету був доступ із коду програми, ієрархію директорій необхідно прописати у так званій змінній

середовища *CLASSPATH* (для Windows). Наприклад, для пакета із назвою `package` *java.awt.image у* змінну середовища *CLASSPATH* потрібно додати запис *java\awt\image.*

Імпорт пакетів

Усі стандартні класи Java зберігаються у пакетах, які мають відповідні імена. Для того, щоб використати клас із наперед визначеного пакету, необхідно вказати його повне ім'я. Для усунення цієї незручності в мові Java визначено оператор *import*, який дозволяє імпортувати окремі класи або цілі пакети.

Для імпорту будь-якого пакету у програму на мові Java використовується наступний синтаксис:

import пакет_1[.пакет_2 . . . [.пакет_N]].Ім'я_Класу;*

Оператор `import` має закінчуватися іменем класу або символом «*», який вказує на те, що імпортувати необхідно весь пакет. Прикладом імпорту пакетів є наступний код.

```
import java.util.Date;
import java.io.*;
```

Якщо у двох імпортованих пакетах є два класи з однаковими іменами, то компілятор видасть помилку тільки після першого звертання до імені класу. У такому випадку необхідно задати повне ім'я пакету із назвою класу в кінці. Наприклад, при створенні об'єкта типу `Vector` отримаємо наступний запис:

```
java.util.Vector v = new java.util.Vector();
```

У мові Java існує пакет, який автоматично імпортується компілятором у всі програми. Це пакет із назвою `java.lang`, який включає 4 інтерфейси, 27 класів, 20 класів помилок і 24 класи виняткових ситуацій. Серед класів пакету `java.lang` є такі важливі класи як `Object`, `String` та `System`, а також класи-оболонки для простих типів.

Обмеження доступу

Класи та пакети є засобами інкапсуляції. Пакети відіграють роль сховищ для класів і вкладених пакетів. В області взаємодії класів та пакетів виокремлюють чотири категорії доступності елементів класу:

1) підкласи у тому ж пакеті;
2) не підкласи у тому ж пакеті;
3) підкласи у різних пакетах;
4) класи, які не є підкласами і знаходяться в інших пакетах.

Усе, що оголошено з модифікатором доступу `public`, доступно з будь-якої точки програми, а все, що оголошено з модифікатором доступу `private` – за межами класу недоступне.

Якщо елементу класу не присвоєно модифікатор доступу, то від доступний у підкласах та інших класах пакету. Такий вид доступу має назву «доступ за замовчуванням» або «дружній доступ». Якщо необхідно, щоб член класу був доступний за межами пакету із підкласів суперкласу, то його необхідно оголосити з модифікатором доступу *protected*.

Створення власних пакетів

Існують наступні рекомендації, щодо використання пакетів у мові Java:

1) кожен клас (за винятком внутрішніх) рекомендується поміщати в окремий файл з іменем класу і розширенням `.java`;

2) рекомендується використовувати кореневий каталог, для усіх створених пакетів;

3) якщо в проект входить понад 5-9 класів, то бажано розміщати їх у підпакетах.

Щоб уникнути конфлікту імен класів, фірма Sun рекомендує розробникам за основу для побудови імен пакетів використовувати уточнене ім'я їхнього домену в Internet, записане у зворотному порядку. Наприклад, для nung.edu.ua, пакети повинні починатися із назви *ua.edu.nung.it.pzas* і т. д.

Клас `Package`

Швидкий розвиток і спеціалізація пакетів спричинив появу у мові Java нового класу *Package* доступного з пакету `java.lang`, який дозволяє отримати інформацією щодо версії пакета. У наступному прикладі використовується клас `Package` з метою отримання інформації про доступні пакети.

```
// Отримання інформації про доступні пакети
class Main {
    public static void main(String[] args) {
        Package pkgs[];

        pkgs = Package.getPackages();
                for (int i=0; i<pkgs.length; i++)
                    System.out.println(pkgs[i].getName() + " " +
        pkgs[i].getImplementationTitle() + " " +
        pkgs[i].getImplementationVendor() + " " +
        pkgs[i].getImplementationVersion());
    }
}
```

Ця програма для кожного з пакетів виводить його ім'я, заголовок, ім'я виробника і номер версії.

Поліморфізм – це здатність екземплярів підкласу грати роль об'єктів батьківського класу, завдяки чому екземпляри підкласу можна використовувати там, де використовуються екземпляри батьківського класу

Якщо два методи з однаковими іменами знаходяться в одному класі, списки параметрів повинні відрізнятися. Такі методи є **перевантажуваними** (overload).

Якщо метод підкласу збігається з методом суперкласу, то метод підкласу **перевизначає** (overriden) метод суперкласу.

Перевизначення методів є основою концепції динамічного зв'язування, що реалізовує **поліморфізм**

Здатність Java робити вибір методу, виходячи з ситуації, називається **динамічним поліморфізмом**. Пошук методу відбувається спочатку в даному класі, потім в суперкласі, поки метод не буде знайдений або не досягнутий **Object.**

Перекривання методів є основою одного з найважливіших понять мови Java – **динамічної диспетчеризації методів** (dynamic method dispatch). Це механізм, який дозволяє викликати перекриті методи не під час компіляції програми, а під час її виконання. Фактично – це реалізація поліморфізму на етапі виконання програми.

Якщо перекритий метод викликається за допомогою посилання на базовий клас, то версія методу визначається в залежності від типу об’єкта, на який вказує це посилання. Іншими словами, визначаючим параметром під час вибору версії перекритого методу є тип об’єкта, на який посилається змінна, а не тип самої змінної. Отже, якщо в базовому класі визначено метод, який надалі перекрито методом у підкласі, то внаслідок посилання змінної базового класу на об’єкти різного типу виконуватимуться різні версії цього методу. Розглянемо вищесказане на поданому нижче прикладі.

У представленому прикладі змінну myOb визначено як змінну класу A, який є базовим класом для класів B та C. Як можна побачити, викликається версія методу call(), визначена типом об’єкту, а не типом посилання myOb.

Перекриття методів дає змогу визначити в базовому класі спільні методи, а в підкласах визначити специфічні шляхи їхньої реалізації. В цьому полягає ще один прояв поліморфізму – один інтерфейс, декілька методів.

Завдяки комбінації наслідування і перекриття методів базовий клас визначає загальний вигляд методів, які надалі використовують усі його підкласи.

Поліморфізм.

Динамічна диспетчеризація методів

```
// Динамічна диспетчеризація методів
class A {
 void call() {
   System.out.println("Метод call() класу A");
 }}

class B extends A {
 // Перевизначення методу call()
 void call() {
   System.out.println("Метод call() класу B");
 }}

class C extends A {
 // Перевизначення методу call()
 void call() {
   System.out.println("Метод call() класу C");
 }}

public class Main {
 public static void main (String args[]) {
  A a = new A();
  B b = new B();
  C c = new C();
  A myOb; // Отримання змінної класу A
  myOb = a; // myOb посилається на об'єкт A
  myOb.call();
  myOb = b; // myOb посилається на об'єкт B
  myOb.call();
  myOb = c; // myOb посилається на об'єкт C
  myOb.call();    }}
```

```
Метод call() класу A
Метод call() класу B
Метод call() класу C
```

Поліморфізм.

Динамічна диспетчеризація методів

```
public class Parent {
public void parentPrint(){
System.out.println("ParentClass");
}}

public class Demo extends Parent {
public void demoPrint (){

System.out.println ("DemoClass");

super.parentPrint();// виклик методу батьківського
}}
```

```
public class Main {

public static void main(String[] args) {
Parent parent = new Parent();
Demo demo = new Demo();
parent.parentPrint();
demo.demoPrint();

}}
```

```
Результат:
ParentClass
DemoClass
ParentClass
```

Поліморфізм.

Динамічна диспетчеризація методів

```
public class Parent {
public void parentPrint(){
System.out.println("ParentClass");
}}

public class Demo extends Parent {
public void demoPrint (){
System.out.println ("DemoClass");
}
public void parentPrint(){
System.out.println("childPrint");
}}
```

```
public class Main {

public static void main(String[] args) {
Parent parent = new Parent();
Demo demo = new Demo();
parent.parentPrint();
demo.demoPrint();
demo.parentPrint();
}}
```

```
Результат:
ParentClass
DemoClass
childPrint
```

Наведемо приклад, в якому продемонструємо доступ до полів з однаковими іменами при наслідуванні:

Поліморфізм. Доступ до полів.

```
class A {
    int x = 1, y = 2;
    public A() {
            y = getX();
            System.out.println("у класі A після
виклику getX() x=" + x + " y=" + y);
            }
    public int getX(){
            System.out.println("у класі A");
            return x;
            } }
```

```
class B extends A {
            int x = 3, y = 4;
            public B() {
            System.out.println("у класі B
x="+x+" y="+y);
            }

    public int getX(){
            System.out.println("у класі B");
            return x;
            } }
```

```
public class DemoAB {
        public static void main (String[] args) {
                A objA = new B();
                B objB = new B();

                System.out.println(objA.x);
                System.out.println(objB.x);
        }}
```

x=1
x=3

Проте скориставшись перетворенням типів вигляду:

((A)objB).x або ((B)objA).x

можна легко дістати доступ до поля x з відповідного класу.

Навіщо потрібні інтерфейси?

- Щоб показати інтерфейс програмування об'єкта (функціональність об'єкта), не показуючи реалізації
  - **це головна мета інкапсуляції**
- Інтерфейс являє собою дещо більше, ніж абстрактний клас, тому що він дозволяє реалізувати «множинне наслідування», яке присутнє в C++.
- Щоб незв'язані класи реалізували однакові методи (поведінку)
  - `один клас не підклас іншого`
- Для створення класу, що реалізує певний інтерфейс (або групу інтерфейсів), використовується ключове слово **implements.**

class Ім'яКласу [extends суперклас] [implements інтерфейс 0 [, інтерфейс1 ...]] { тіло класу }

Інтерфейси можуть бути оголошені з рівнем доступу ***public*** (загальнодоступні) або з рівнем доступу без задання (доступні лише всередині пакету).

```
// Файл, який містить цей код, повинен мати назву Holovnuj.java
interface Dryhorjadnuj {
  ...
}
interface WqeOdun {
  ...
}
public interface Holovnuj {
/* загальнодоступним (public) може бути тільки один інтерфейс
або клас у файлі, назва файлу має повністю збігатися із назвою такого інтерфейсу або
класу, враховуючи регістр букв */
  ...
}
```

Оскільки, інтерфейс містить тільки оголошення без реалізації, то створити екземпляр інтерфейсу заборонено.

- **До Java 8 інтерфейси могли мати лише публічні абстрактні методи.** Не можливо додати нову функціональність до наявного інтерфейсу, не змушуючи всіх класів реалізації створювати реалізацію нових методів, також не було можливо створити методи інтерфейсу з реалізацією.
- **Java 8 представила до таблиці кілька абсолютно нових функцій, включаючи** статичні та методи за замовчуванням в інтерфейсах.
- **Методи за замовчуванням (типові) в інтерфейсах:**
  - оголошуються ключовим словом **default** на початку підпису методу , і вони забезпечують реалізацію:
- **дозволяють** нам додавати **нові методи** в інтерфейс, який автоматично доступний у реалізаціях . Таким чином, не потрібно змінювати класи реалізації.
- **Вони доступні через екземпляр** класу реалізації та можуть бути перевизначені.

```
1public interface MyInterface {
2
3   // regular interface methods
4
5   default void defaultMethod() {
6      // default method implementation
7   }
8}
```

```
// Використання інтерфейсів та абстрактних класів
interface  A {
   void call1();
   void call2();
}
 // Наслідування інтерфейсом B інтерфейсу A
interface  B extends A {
   void call3();
}
// Реалізація абстрактним класом C інтерфейсу A
abstract class C implements A {
   public void call1() {
   System.out.println("Метод call1() реалізований у класі C");
   }
   abstract void call4();
}
// Наслідування класом D абстрактного класу C та реалізація інтерфейсу B
class D extends C implements B {
   public void call2() {
         System.out.println("Метод call2() реалізований у класі D");
   }
   public void call3() {
         System.out.println("Метод call3() реалізований у класі D");
   }
   void call4() {
         System.out.println("Метод call4() реалізований у класі D");}}
public class Main {
   public static void main (String args[]) {
```

```
        D d = new D();

        A a = d; // Присвоєння змінній a посилання на об'єкт D
        a.call1();
        a.call2();
        // a.call3(); // Заборонено, у A не визначено метод call3()
        // a.call4(); // Заборонено, у A не визначено метод call4()
        System.out.println();

        B b = d; // Присвоєння змінній b посилання на об'єкт D
        b.call1();
        b.call2();
        b.call3();
        // b.call4(); // Заборонено, у B не визначено метод call4()
        System.out.println();
        C c = d; // Присвоєння змінній c посилання на об'єкт D
        c.call1();
        c.call2();
        // c.call3(); // Заборонено, у C не визначено метод call3()
        c.call4();
        System.out.println();

        d.call1();
        d.call2();
        d.call3();
        d.call4();
    }
    }
    Вивід:
    Метод call1() реалізований у класі C
    Метод call2() реалізований у класі D
     Метод call1() реалізований у класі C
    Метод call2() реалізований у класі D
    Метод call3() реалізований у класі D
     Метод call1() реалізований у класі C
    Метод call2() реалізований у класі D
    Метод call4() реалізований у класі D
     Метод call1() реалізований у класі C
    Метод call2() реалізований у класі D
    Метод call3() реалізований у класі D
  Метод call4() реалізований у класі D
```

Як можна бачити з представлено прикладу один інтерфейс може наслідувати методи іншого інтерфейсу, при цьому у класі, що реалізує такий інтерфейс повинні бути реалізовані методи як базового інтерфейсу, так і підінтерфейсу. Якщо клас по якійсь із причин не реалізує один або декілька методів інтерфейсу, то він повинен бути оголошений з ключовим словом abstract.

Статичні поля, методи і поліморфізм.

1. Для них проблем і конфліктів, пов'язаних з поліморфізмом, не існує.
2. Статичні методи і статичні поля належать класу і поява наслідників на них не впливає:

```
 class Parent {
static int a=2;
}
class Child extends Parent {
 static int a=3;
}
```

Який буде результат?

```
Child c = new Child();
System.out.println(c.a);
Parent p = c;
System.out.println(p.a);
```

Аналогічно:

```
System.out.println(Child.a)
System.out.println(Parent.a)
```

Неважливо, на який об'єкт вказує посилання. Більш того, вона може бути навіть дорівнює null. Все визначається типом посилання.

Яка ситуація з методами:

```
class Parent {
static public int getValue() {
    return 0;   }}
class Child extends Parent {
 static public int getValue() {
 return 1;
}}
```

Який буде результат?

```
Child c = new Child();
System.out.println(c.getValue());
Parent p = c;
System.out.println(p.getValue());
```

Аналогічно:

```
System.out.println(Child.getValue());
System.out.println(Parent.getValue());
```

Результатом будет: 1
0

Визначається хто з методів буде викликаний за типом посилання, а не за типом екземпляра класу.

Попередньо йшлося про перетворення об'єктів найпростіших типів. Проте з об'єктами класів все відбувається трохи інакше. Допустимо, у нас є наступна ієрархія класів:

```
public class Program{

   public static void main(String[] args) {

      Person tom = new Person("Tom");
      tom.display();
      Person sam = new Employee("Sam", "Oracle");
      sam.display();
      Person bob = new Client("Bob", "DeutscheBank", 3000);
      bob.display();
   }
}
// клас людини
```

```
class Person {
     private String name;
    public String getName() { return name; }
    public Person(String name){
            this.name=name;
   }

   public void display(){
         System.out.printf("Person %s \n", name);
   }
}
// представнник деякої компанії
class Employee extends Person{
    private String company;
   public Employee(String name, String company) {
      super(name);
      this.company = company;
   }
   public String getCompany(){ return company; }
      public void display(){
     System.out.printf("Employee %s works in %s \n", super.getName(), company);
   }
}
// клас клієнта банку
class Client extends Person{

   private int sum; // Змінна для збереження суми на рахунку
   private String bank;

   public Client(String name, String bank, int sum) {

      super(name);
      this.bank=bank;
      this.sum=sum;
   }

   public void display(){

      System.out.printf("Client %s has account in %s \n", super.getName(), bank);
   }

   public String getBank(){ return bank; }
   public int getSum(){ return sum; }
}
```

У цій ієрархії класів можна прослідкувати наступний ланцюг успадкування: Object (всі класи неявно успадковуються від типу Object) -> Person -> Employee | Client.

Суперкласи зазвичай розміщуються вище підкласів, тому на вершині успадкування знаходиться клас Object, а в самому низу Employee та Client.

Об'єкт підкласу також є об'єктом суперкласу. Тому в програмі ми можемо написати так:

```
Object tom = new Person("Tom");
Object sam = new Employee("Sam", "Oracle");
Object kate = new Client("Kate", "DeutscheBank", 2000);
Person bob = new Client("Bob", "DeutscheBank", 3000);
Person alice = new Employee("Alice", "Google");
```

Це так зване висхідне перетворення (від підкласу внизу до суперкласу вгорі ієрархії) або upcasting. Таке перетворення здійснюється автоматично.

Зворотне не завжди вірне. Наприклад, об'єкт Person який завжди є об'єктом Employee чи Client. Тому низхідне перетворення або downcasting від суперкласу до підкласу автоматично не виконується. І тут нам треба використовувати операцію перетворення типів.

```
Object sam = new Employee("Sam", "Oracle");

// низхідне перетворення від Object до типу Employee
Employee emp = (Employee)sam;
emp.display();
System.out.println(emp.getCompany());
```

У разі змінна sam приводиться до типу Employee. І потім, через об'єкт emp ми можемо звернутися до функціоналу об'єкта Employee.

Ми можемо перетворити об'єкт Employee по всій прямій лінії наслідування від Object до Employee.

Приклади спадних перетворень:

```
Object kate = new Client("Kate", "DeutscheBank", 2000);
((Person)kate).display();

Object sam = new Employee("Sam", "Oracle");
((Employee)sam).display();
```

Але розглянемо ще одну ситуацію:

```
Object kate = new Client("Kate", "DeutscheBank", 2000);
Employee emp = (Employee) kate;
emp.display();

// або так
((Employee)kate).display();
```

У разі змінна типу Object зберігає посилання об'єкт Client. Ми можемо без помилок привести цей об'єкт до типів Person чи Client. Але при спробі перетворення типу Employee ми отримаємо помилку під час виконання. Оскільки kate не є об'єктом типу Employee.

***Пізнє зв'язування* (*late binding*):**

- (динамічна диспетчеризація методів) – механізм, за допомогою якого рішення до якого методу звернутися при перевизначенні здійснюється під час виконання, а не під час компіляції.
- Для всіх методів java використовується механізм пізнього зв'язування
- Механізм не працює, якщо метод був оголошений як **final** (приватні методи є **final** за замовчуванням).

Висновки

У представленому матеріалі розглянуто механізм імплементації, що полягає у реалізації класом необмеженої кількості інтерфейсів. Інтерфейси та множинне наслідування забезпечують реалізацію ще однієї концепції ООП, яка проявляється у формі динамічного поліморфізму. Перекриття методів дає змогу визначити в базовому класі спільні методи, а в підкласах визначити специфічні шляхи їхньої реалізації.

Контрольні запитання

1. Яким чином проявляється концепція поліморфізму у мові Java?
2. У яких випадках відбувається перекриття методів?
3. В чому полягає суть динамічної диспетчеризації методів?
4. Яким чином у мові Java реалізується множинне наслідування?
5. Який синтаксис створення інтерфейсу?
6. Який синтаксис реалізації класом одного або декількох інтерфейсів?
7. В чому суть низхідного та висхідного приведення об’єктів.
8. В чому полягає суть методики пізнього зв’язування?